MyKad

Complete Technical Reference

The Ultimate A–Z Developer Guide to the Malaysian National Identity Card

Version 3.0 | April 2026 | Author: (deadboy18)

https://github.com/deadboy18/mykad-reader

FOR AUTHORISED DEVELOPMENT USE ONLY

Smart Card Chip Architecture • IC Number Format & Checksum • APDU Commands • Byte-Level Parsing • Full Code in Python, C, Go, JavaScript, PHP, C#, Java • Server API • AI/OCR Integration • Security & PDPA Compliance

Table of Contents

1. Overview — What is MyKad
2. Physical Card Specifications & Security Features
3. IC Number Format (YYMMDD-PB-###G)
4. Place of Birth (PB) Code Directory — Complete
5. Checksum Algorithm — ISO 7064 Mod 11,2 (Deep Dive)
6. Checksum, Gender & Sequence Assignment Logic
7. Sample IC Numbers — Worked Examples (Male & Female)
8. Smart Card Chip Architecture
9. Hardware Requirements & Troubleshooting
10. APDU Command Reference
11. JPN File Structure & Byte-Level Parsing
12. Data Type Decoding
13. Complete Extractable Data Reference
14. Implementation: Python (Full CLI Tool)
15. Implementation: C / WinSCard (Low-Level)
16. Implementation: Go (Struct-Based)
17. Implementation: Node.js (OpenSC)
18. Cross-Language Checksum Validators (JS, Python, C#, PHP, Go, Java, C)
19. Server API — Flask REST + WebSocket Architecture
20. AI/OCR ID Card Scanning Integration
21. Database Schema (SQLite & SQL)
22. Security, PDPA Compliance & Best Practices
23. Reference Resources

1. Overview — What is MyKad

MyKad (Malaysia Kad) is the compulsory national identity card issued by Jabatan Pendaftaran Negara (JPN — National Registration Department) to all Malaysian citizens aged 12 and above, and to permanent residents (MyPR) and temporary residents (MyKAS). It replaced the older laminated High-Quality Identity Card in 2001.

The MyKad is a multi-application smart card that serves as proof of identity, citizenship status, driving licence, health information carrier, and e-wallet (Touch ‘n Go). It contains a contact chip (ISO 7816) for identity data and a separate contactless RFID chip (ISO 14443-3A) for the e-wallet.

1.1 What’s on the Contact Chip

- Personal identity data (name, IC number, DOB, gender, race, religion, nationality)
- JPEG facial photograph (~15KB, stored in 4000-byte allocation)
- Fingerprint templates (right and left thumb, proprietary IRIS Corp format)
- Residential address (3 address lines, postcode, city, state)
- SOCSO / PERKESO number
- JPJ driving licence data (separate application on same chip)
- Immigration / passport references (separate application on same chip)

1.2 What’s on the Contactless RFID Chip

Touch ‘n Go e-wallet stored value only. This is a completely separate Mifare Classic 1K chip (ISO 14443-3A) with its own sector keys. It is NOT accessible via the contact interface. NFC readers like ACR122U can only see this chip.

1.3 Card Variants

Card Type	Issued To	Notes
MyKad	Malaysian citizens aged 12+	Standard 12-digit IC number
MyKid	Malaysian citizens under 12	Same format, issued to minors
MyPR	Permanent residents	Same format, assumed same checksum
MyKAS	Temporary residents	Same format, less tested with checksum
MyTentera	Military personnel	Variant with military extensions

2. Physical Card Specifications & Security Features

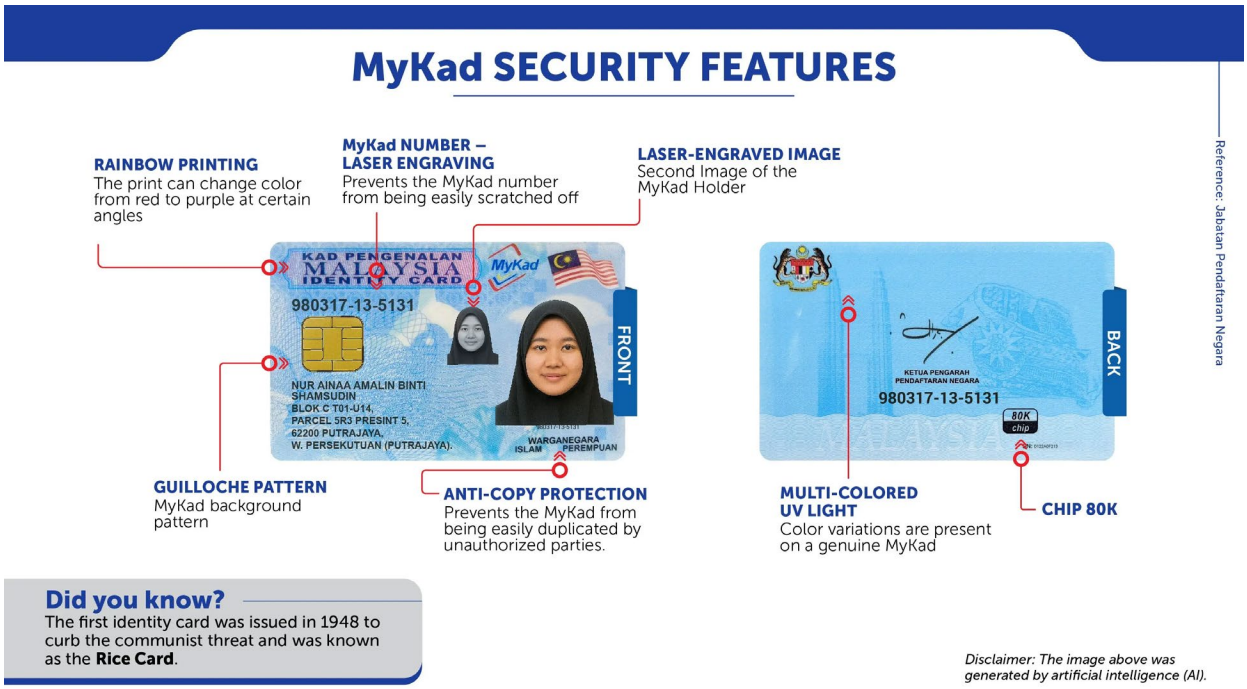

Figure 1: MyKad Security Features — Front and Back (AI-generated reference image)

Property	Value
Card Standard	ID-1 (ISO/IEC 7810) — credit card size
Dimensions	85.6 mm × 53.98 mm × 0.76 mm
Contact Chip	ISO 7816 compliant, 80K EEPROM
Contactless Chip	Mifare Classic 1K (ISO 14443-3A) for TnG
Material	Polycarbonate with laser-engraved data

2.1 Security Features

Rainbow Printing: Print changes colour from red to purple at different angles.

Laser Engraving: MyKad number is laser-engraved into the card surface to prevent tampering.

Ghost Image: Second laser-engraved photo of the holder on the back of the card.

Guilloche Pattern: Complex interlocking background pattern, extremely difficult to reproduce.

Anti-Copy Protection: Features that prevent photocopying or scanning duplication.

Multi-Coloured UV Light: Colour variations visible under ultraviolet light.

80K Chip: Contact chip on back with 80KB EEPROM storage.

2.2 Printed Information

Front: Title “KAD PENGENALAN MALAYSIA / IDENTITY CARD”, MyKad logo, coat of arms, photo, full name, address, IC number (YYMMDD-PB-####), nationality, religion, gender.

Back: Ketua Pengarah Pendaftaran Negara signature, IC number (repeated), 80K chip pad, ghost image, serial number.

2.3 Historical Note

Malaysia’s first identity card was issued in 1948 to curb the communist threat and was known as the “Rice Card.”

3. IC Number Format (YYMMDD-PB-###G)

The MyKad identification number is a 12-digit numeric string. On the physical card it is formatted with dashes: YYMMDD-PB-####. In the chip and databases it is stored as 12 continuous digits.

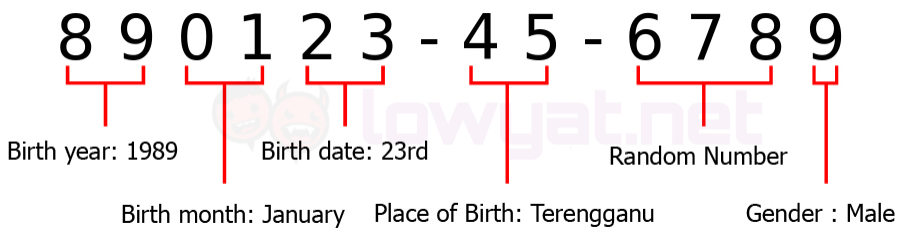

Figure 2: MyKad IC Number Structure — YYMMDD-PB-###G (Source: Lowyat.net)

3.1 Structure Breakdown

Position	Digits	Field	Description
1–2	YY	Birth Year	Last two digits of birth year
3–4	MM	Birth Month	01–12
5–6	DD	Birth Day	01–31
7–8	PB	Place of Birth	State code (domestic) or country code (foreign)
9–11	###	Sequence Number	Sequential number assigned by JPN's system
12	G	Gender + Checksum	Odd = Male, Even = Female. Also the checksum digit.

3.2 Century Determination

Since YY is only two digits, the century is inferred from the first digit of the sequence number (position 9):

Seq First Digit	Century	Year Range	Example
0, 1, 2, 3, 4	2000s	2000–2099	YY=05, seq=023x → 2005
5, 6, 7, 8, 9	1900s	1900–1999	YY=89, seq=678x → 1989

3.3 Display Format

```
Printed on card:  980317-13-5131
Stored in chip:   980317135131 (12 digits, no dashes)
Stored in DB:     980317135131
```

4. Place of Birth (PB) Code Directory — Complete

The 2-digit PB code at positions 7–8 identifies where the holder was born.

4.1 Malaysian States — Primary Codes

Code	State / Territory
01	Johor
02	Kedah
03	Kelantan
04	Melaka (Malacca)
05	Negeri Sembilan
06	Pahang
07	Pulau Pinang (Penang)
08	Perak
09	Perlis
10	Selangor
11	Terengganu
12	Sabah
13	Sarawak
14	Wilayah Persekutuan Kuala Lumpur
15	Wilayah Persekutuan Labuan
16	Wilayah Persekutuan Putrajaya

4.2 Malaysian States — Legacy / Alternate Codes

Code Range	State
21–24	Johor
25–27	Kedah
28–29	Kelantan
30	Melaka
31, 59	Negeri Sembilan
32–33	Pahang
34–35	Pulau Pinang
36–39	Perak
40	Perlis
41–44	Selangor
45–46	Terengganu
47–49	Sabah

50–53	Sarawak
54–57	Kuala Lumpur
58	Labuan

4.3 Foreign Born — Country / Region Codes

Code	Country / Region
60	Brunei
61	Indonesia
62	Cambodia
63	Laos
64	Myanmar
65	Philippines
66	Singapore
67	Thailand
68	Vietnam
71–72	Born outside Malaysia prior to 2001
74	China
75	India
76	Pakistan
77	Saudi Arabia
78	Sri Lanka
79	Bangladesh
82	Unknown state
83	Asia-Pacific (Australia, New Zealand, Pacific Islands)
84	South America (Argentina, Brazil, Chile, Colombia, Peru, etc.)
85	Africa (Algeria, Angola, Cameroon, Egypt, Ghana, Kenya, Nigeria, South Africa, etc.)
86	Europe (Austria, Belgium, Cyprus, Denmark, Finland, France, Germany, Greece, Italy, Luxembourg, Malta, Monaco, Netherlands, Norway, Portugal, Spain, Sweden, Switzerland, UK, etc.)
87	Britain / Great Britain / Ireland
88	Middle East (Bahrain, Iran, Iraq, Jordan, Kuwait, Lebanon, Oman, Qatar, Syria, Turkey, UAE, Israel, etc.)
89	Far East (Japan, North Korea, South Korea, Taiwan)
90	Caribbean (Bahamas, Barbados, Cuba, Dominican Republic, Haiti, Jamaica, Mexico, Nicaragua, Panama, Puerto Rico, Trinidad and Tobago, etc.)
91	North America (Canada, Greenland, USA)

92	Soviet Union / USSR (Albania, Belarus, Bosnia, Bulgaria, Croatia, Czech Republic, Estonia, Georgia, Hungary, Latvia, Lithuania, Montenegro, Poland, Romania, Russia, Serbia, Ukraine, etc.)
93	Other Countries (Afghanistan, Andorra, Bhutan, Hong Kong, Iceland, Macau, Maldives, Mongolia, Nepal, etc.)
98	Stateless Person (Article 1/1954)
99	Unspecified Nationality / Refugee / Mecca / Neutral Zone

5. Checksum Algorithm — ISO 7064 Mod 11,2 (Deep Dive)

JPN has never officially confirmed any checksum algorithm for MyKad numbers. However, through extensive community testing (see Bryan Lam / beeell1's malaysian-ic-number-validator on GitHub), the ISO 7064 Mod 11,2 algorithm has been validated against a large corpus of real IC numbers with a near-perfect success rate.

This is the same algorithm used in China's Resident Identity Card Number (18-digit, with 'X' as a possible check character).

5.1 Algorithm Definition

Input: The first 11 digits of the IC number (positions 1–11).

Output: A single check digit (0–9), or 'X' (representing 10, which is skipped for MyKad).

Step-by-Step

1. Reverse the 11-digit string into an array.
2. For each position i (starting from 2), compute the weight: W(i) = 2^(i-1) mod 11.
3. Multiply each reversed digit by its weight W(j+2) for j = 0..10.
4. Sum all the products to get S.
5. Compute checkDigit = (12 − (S mod 11)) mod 11.
6. If checkDigit is 0–9, it is the valid 12th digit. If 10, this sequence is impossible and is skipped.

5.2 Weight Table

j (rev. position)	Weight Index i=j+2	W(i) = 2^(i-1) mod 11
0	2	2
1	3	4
2	4	8
3	5	5
4	6	10
5	7	9
6	8	7
7	9	3
8	10	6
9	11	1
10	12	2

5.3 Detailed Walkthrough Example

Validate IC number: 850101135029 (sample 7.1)

```
First 11 digits: 85010113502
Reversed array:  [2, 0, 5, 3, 1, 1, 0, 1, 0, 5, 8]
Weights:         [2, 4, 8, 5, 10, 9, 7, 3, 6, 1, 2]

Multiply each pair:
  2×2=4, 0×4=0, 5×8=40, 3×5=15, 1×10=10,
  1×9=9, 0×7=0, 1×3=3, 0×6=0, 5×1=5, 8×2=16

Sum S = 4+0+40+15+10+9+0+3+0+5+16 = 102
S mod 11 = 102 mod 11 = 3
checkDigit = (12 - 3) mod 11 = 9 mod 11 = 9

Expected last digit: 9
Actual last digit:   9 → CHECKSUM VALID ✓
Gender: 9 is odd → Male ✓
```

Now let’s see what happens when a sequence is impossible:

```
Try IC prefix 85010113 + seq 005:
First 11: 85010113005
Sum S = 68, S mod 11 = 2
checkDigit = (12 - 2) mod 11 = 10
checkDigit = 10 → Would need 'X' → IMPOSSIBLE SEQUENCE, system skips it.
```

6. Checksum, Gender & Sequence Assignment Logic

The 12th digit serves dual purpose: it is both the ISO 7064 checksum AND the gender indicator (odd = male, even = female). This works because JPN's system iterates sequences until both conditions are met.

6.1 The Assignment Algorithm (Theory)

7. Fix the prefix: YYMMDD (6 digits) + PB (2 digits) = 8 fixed digits.
8. Start from sequence 001 (or next unassigned).
9. Compute checksum for the 11-digit candidate (prefix + 3-digit sequence).
10. If checksum = 10 ('X'): skip, try next sequence.
11. If checksum parity doesn't match applicant's gender: skip.
12. If checksum digit matches gender parity AND sequence not previously assigned: ASSIGN this IC number.

6.2 Worked Example

Person: Male, born 1 January 2000, Sarawak (PB=13).

```
Prefix: 000101-13-

Seq 001: 00010113001 → checksum=6 (even=Female) → SKIP
Seq 002: 00010113002 → checksum=4 (even=Female) → SKIP
Seq 003: 00010113003 → checksum=2 (even=Female) → SKIP
Seq 004: 00010113004 → checksum=0 (even=Female) → SKIP
Seq 005: 00010113005 → checksum=9 (odd=Male)   → ASSIGN → 000101130059
```

So the final IC number is 000101130059.

6.3 Why ~1/11 Sequences Are Impossible

Mod 11 can produce 0–10. Result 10 would need 'X' (as China uses). Since MyKad is digits-only, these sequences are simply discarded. This means roughly 9% of all possible sequences will never appear in real IC numbers.

6.4 Implications for Developers

- About half of valid sequences will produce odd check digits (male) and half even (female).
- The checksum detects single-digit transcription errors with ~90% accuracy.
- Gender can be verified from the IC number alone without accessing the chip.
- A "valid" number passing checksum does NOT mean it's assigned to a real person.

7. Sample IC Numbers — Worked Examples

The following are fictitious demonstration records. None correspond to real persons. All checksum digits are computed using the ISO 7064 Mod 11,2 algorithm documented above.

DISCLAIMER: These are FAKE identities created for documentation purposes only. Any resemblance to real persons is entirely coincidental.

7.1 Sample: Malaysian Male (Sarawak, 1985)

Field	Value	Explanation
IC Number	850101135029	Verified: passes ISO 7064 Mod 11,2
Formatted	850101-13-5029	
YY	85	Birth year last 2 digits
MM	01	January
DD	01	1st
PB	13	Sarawak
Sequence	502	Seq first digit = 5 ≥ 5 → 1900s century
Century	1900s	1900 + 85 = 1985
Checksum	9	Odd = Male ✓
Full DOB	1985-01-01	
Gender	Male	Last digit 9 is odd
Name (demo)	AHMAD BIN ABDULLAH	Fictitious
Address (demo)	NO 23 JALAN MASJID, 93000 KUCHING, SARAWAK	Fictitious
Race (demo)	Malay	
Religion (demo)	Islam	
Nationality	WARGANEGARA	Citizen

7.2 Sample: Malaysian Female (Kuala Lumpur, 1998)

Field	Value	Explanation
IC Number	980317145002	Verified: passes ISO 7064 Mod 11,2
Formatted	980317-14-5002	
YY	98	
MM	03	March
DD	17	17th
PB	14	Kuala Lumpur
Sequence	500	Seq first digit = 5 ≥ 5 → 1900s century
Century	1900s	1900 + 98 = 1998

Full DOB	1998-03-17	
Checksum	2	Even = Female ✓
Gender	Female	Last digit 2 is even
Name (demo)	NUR AINAA BINTI SHAMSUDIN	Fictitious
Address (demo)	BLOK C T01-U14, PARCEL 5R3, 62200 PUTRAJAYA	Fictitious
Race (demo)	Malay	
Religion (demo)	Islam	

7.3 Sample: Malaysian Male (Penang, Chinese, 1990)

Field	Value	Explanation
IC Number	900523075049	Verified: passes ISO 7064 Mod 11,2
Formatted	900523-07-5049	
Full DOB	1990-05-23	Seq first digit = 5 ≥ 5 → 1900s
PB	07	Pulau Pinang
Sequence	504	
Checksum	9	Odd = Male ✓
Gender	Male	
Name (demo)	TAN WEI MING	Fictitious
Address (demo)	12A LORONG BURMA, 10250 GEORGETOWN, PULAU PINANG	Fictitious
Race (demo)	Chinese	
Religion (demo)	Buddhist	

7.4 Sample: Malaysian Female (Selangor, Indian, 2001)

Field	Value	Explanation
IC Number	010517100004	Verified: passes ISO 7064 Mod 11,2
Formatted	010517-10-0004	
Full DOB	2001-05-17	Seq first digit = 0 < 5 → 2000s
PB	10	Selangor
Sequence	000	First available sequence for this DOB+PB
Checksum	4	Even = Female ✓
Gender	Female	
Name (demo)	PRIYA A/P KRISHNAN	Fictitious

Address (demo)	45 JALAN SS2/55, 47300 PETALING JAYA, SELANGOR	Fictitious
Race (demo)	Indian	
Religion (demo)	Hindu	
Nationality	WARGANEGARA	

7.5 Sample: Foreign Born Male (Singapore, 1988)

Field	Value	Explanation
IC Number	880715665005	Verified: passes ISO 7064 Mod 11,2
Formatted	880715-66-5005	
Full DOB	1988-07-15	Seq first digit = 5 ≥ 5 → 1900s
PB	66	Singapore
Sequence	500	
Checksum	5	Odd = Male ✓
Gender	Male	
Name (demo)	RAMASAMY A/L MUTHU	Fictitious
Nationality	WARGANEGARA	Malaysian citizen, born in Singapore

7.6 Checksum Verification for Each Sample

Every sample above has been verified with the ISO 7064 Mod 11,2 algorithm. Developers can copy any of these IC numbers into their validator and confirm they pass. Here is the full computation for sample 7.1:

```
IC: 850101135029
First 11: 85010113502
Reversed: [2, 0, 5, 3, 1, 1, 0, 1, 0, 5, 8]
Weights:  [2, 4, 8, 5, 10, 9, 7, 3, 6, 1, 2]
Products: [4, 0, 40, 15, 10, 9, 0, 3, 0, 5, 16]
Sum S = 102
S mod 11 = 102 mod 11 = 3
checkDigit = (12 - 3) mod 11 = 9
Expected: 9 | Actual last digit: 9 → MATCH ✓
Gender: 9 is odd → Male ✓
Century: seq first digit 5 ≥ 5 → 1900s, 1900+85 = 1985 ✓
```

All 5 samples follow the same pattern: the checksum matches, the gender parity is correct, and the century rule is consistent.

8. Smart Card Chip Architecture

8.1 Dual-Chip Design

Chip	Standard	Interface	Storage	Purpose
Contact	ISO 7816-1/2/3/4	Gold pad	80KB EEPROM	Identity, photo, fingerprints, address, licence, immigration
Contactless	ISO 14443-3A	RFID antenna	1KB Mifare Classic	Touch ‘n Go e-wallet only

8.2 Contact Chip Applications (AIDs)

App	AID (hex)	ASCII	Contents
JPN	A0 00 00 00 74 4A 50 4E 00 10	JPN	Identity, photo, fingerprints, address, SOCSO, locality
JPJ	A0 00 00 00 74 4A 50 4A 00 10	JPJ	Driving licence data
IMM	A0 00 00 00 74 49 4D 4D 00 10	IMM	Immigration / passport references

8.3 AID Structure

```
A0 00 00 00 74  4A 50 4E  00 10
|-- RID ---|  |App Name|  |Sfx|
IRIS Corp     J  P  N
```

8.4 Interface Standards

Standard	Interface	Usage
ISO 7816-1/2/3	Contact	Physical and electrical interface
ISO 7816-4	Contact	APDU command/response, file navigation
ISO 14443-3A	Contactless	TnG e-wallet (Mifare Classic 1K)
PC/SC (CCID)	USB	OS-level smart card reader API

9. Hardware Requirements & Troubleshooting

9.1 Reader Requirements

Any USB smart card reader that is PC/SC compliant (CCID class), supports ISO 7816 contact interface, and appears in SCardListReaders().

9.2 Tested Readers

Device	Tested	Notes
PS/SC CCID ISO7816 USB Reader (Admire IT)	YES	Most common in Malaysia
ACR39U Smart Card Reader (ACS)	YES	PocketMate micro-USB variant also works
ACS ACR38U / ACR38-CCID	YES	Older but fully compatible
Any PC/SC CCID reader	Likely	Must appear in SCardListReaders()
NFC-only (ACR122U, PN532)	NO	Reads TnG RFID only, CANNOT read contact chip

9.3 OS Setup

Windows

PC/SC is built-in. Ensure “Smart Card” service is running (services.msc). Most readers are plug-and-play CCID.

Linux

```
sudo apt install pcscd libpcsclite-dev
sudo systemctl start pcscd
```

macOS

PC/SC supported natively via CryptoTokenKit.

Verify Detection

```
opensc-tool --list-readers
```

9.4 Troubleshooting Matrix

Symptom	Likely Cause	Fix
Reader not found	PC/SC service not running	Start Smart Card service; install reader driver
Card timeout / no response	Dirty or oxidised chip contacts	Clean gold contacts with isopropyl alcohol and soft cloth
Partial data / corruption	Reader returns fewer bytes than requested	Use Fix 3: decrement by actual bytes received, not chunk size
Crash during photo read	252-byte chunk too large for older chips	Use Fix 2: cap chunk at 0xF0 (240 bytes), not 0xFC

SW 6A 80 error	Wrong Set Length format (file ID in payload)	Use Fix 1: constant 08 00 00 payload in Set Length
SW 69 83 error	Card permanently blocked	3 failed write attempts bricked it. Card is dead. Get new MyKad from JPN.
SW 6A 82 error	File not found	Wrong file number or app not selected. Call select_jpn() first.
Card works in one reader, not another	Reader incompatibility	Try a different USB port or reader model
Intermittent read failures	Loose card contact	Ensure card is fully inserted and flat against contacts

10. APDU Command Reference

APDU (Application Protocol Data Unit) is the communication format defined by ISO 7816-4.

10.1 The Five Commands

Command	APDU (hex)	Purpose
Select App	00 A4 04 00 0A [AID]	Select JPN/JPJ/IMM application
Get Response	00 C0 00 00 05	Confirm app selection
Set Length	C8 32 00 00 05 08 00 00 [LenLo] [LenHi]	Declare read length
Select Info	CC 00 00 00 08 [FileNo] 00 01 00 [OffLo] [OffHi] [LenLo] [LenHi]	Point to file + offset
Read Info	CC 06 00 00 [ChunkLen]	Pull data (max 0xF0=240 per call)

All multi-byte offset and length values are little-endian.

10.2 Select JPN Sequence

```
Reader → 00 A4 04 00 0A A0 00 00 00 74 4A 50 4E 00 10
Card   → 61 05
Reader → 00 C0 00 00 05
Card   → 6F 03 82 01 38 90 00
```

10.3 Read File Sequence

```
1. Set Length:  C8 32 00 00 05 08 00 00 [LenLo] [LenHi]
   CRITICAL: '08 00 00' is constant. Do NOT put file ID here.
2. Select Info: CC 00 00 00 08 [FileNo] 00 01 00 [OffLo] [OffHi] [LenLo] [LenHi]
3. Read Info:   CC 06 00 00 [ChunkLen] → loop in 240-byte chunks
```

10.4 Status Words

SW	Meaning
90 00	SUCCESS
61 XX	More data available — send Get Response
6A 80	Wrong data (bad Set Length format)
6A 82	File not found
6A 86	Incorrect P1/P2
6D 00	INS not supported
6E 00	CLA not supported (wrong app)
69 83	Card PERMANENTLY BLOCKED

10.5 Hardware Fixes (Critical)

Fix 1: Set Length uses constant 08 00 00. Old code with file ID causes 6A 80.

Fix 2: Max chunk 0xF0 (240), not 0xFC (252). Older chips crash at 252.

Fix 3: Decrement remaining by actual bytes received, not requested chunk size.

11. JPN File Structure & Byte-Level Parsing

All data: fixed-width, space-padded (0x20), no separators. Every file starts with a 3-byte header; data begins at offset 0x0003.

11.1 File Directory

File	Hex	Header	Data Size	Contents
1	0x01	01 04 24	421 B (0x01A5)	Personal identity
2	0x02	01 40 03	4000 B (0x0FA0)	JPEG photo
3	0x03	01 12 03	~1216 B	Fingerprint templates
4	0x04	01 01 52	148 B (0x0094)	Address
5	0x05	01 12 00	9 B	SOCSO number
6	0x06	—	10 B (0x0A)	Locality / mukim

11.2 File 1 — Personal Identity (0x01)

Offsets relative to read buffer from file offset 0x0003:

Offset	Len	SDK Name	Description
0x00–0x95	150	JPN_OrgName	Full name (ASCII, space-padded)
0x96–0xE5	80	JPN_GMPCName	GMPC name (multi-line)
0xE6–0x10D	40	JPN_KPTName	KPT name (multi-line)
0x10E–0x11A	13	JPN_IDNum	IC number (12 digits + space)
0x11B	1	JPN_Gender	'L'=Male, 'P'=Female
0x11C–0x123	8	JPN_OldIDNum	Old IC (spaces if none)
0x124–0x127	4	JPN_BirthDate	DOB packed BCD (YYYYMMDD)
0x128–0x140	25	JPN_BirthPlace	Birth place
0x141–0x144	4	JPN_DateIssued	Issue date packed BCD
0x145–0x156	18	JPN_Citizenship	Nationality
0x157–0x16F	25	JPN_Race	Race
0x170–0x17A	11	JPN_Religion	Religion
0x17B	1	JPN_EastMalaysian	East Malaysian flag
0x180–0x18A	11	JPN_OtherID	Other ID (PR number etc.)
0x18C	1	JPN_CardVer	Card version (usually 0x02)
0x18D–0x190	4	JPN_GreenCardExpiry	PR expiry (packed BCD)
0x191–0x1A4	20	JPN_GreenCardNat	PR nationality

11.3 File 2 — Photo (0x02)

4000 bytes allocated, actual JPEG shorter. Find last FF D9 (JPEG EOI) and slice at position + 2.

11.4 File 3 — Fingerprints (0x03)

Offset	Length	Description
0x00–0x13	20 B	Metadata: “R1L1” + zero bytes
0x14–0x0269	598 B	Right thumb (proprietary)
0x026A–0x04BF	598 B	Left thumb (proprietary)

WARNING: Proprietary IRIS Corp format. Store raw bytes only. Matching requires official SDK.

11.5 File 4 — Address (0x04)

Offset	Len	SDK Name	Description
0x00–0x1D	30	JPN_Address1	Address line 1
0x1E–0x3B	30	JPN_Address2	Address line 2
0x3C–0x59	30	JPN_Address3	Address line 3
0x5A–0x5C	3	JPN_Postcode	Postcode (packed BCD)
0x5D–0x75	25	JPN_City	City name
0x76–0x93	30	JPN_State	State name

11.6 File 5 (SOCSO) & File 6 (Locality)

File 5: 9-byte ASCII SOCSO/PERKESO number. File 6: 10-byte ASCII locality/mukim. Both space-padded.

12. Data Type Decoding

12.1 ASCII String

```
Python: raw_bytes.decode('ascii', errors='ignore').strip()
C:      memcpy(buf, raw, len); buf[len]='\0'; /* then trim trailing 0x20 */
Go:     strings.TrimRight(string(raw), " ")
```

12.2 Packed BCD Date (4 bytes → YYYY-MM-DD)

```
Raw: 20 01 05 30 → hex "20010530" → 2001-05-30
Python: h = raw.hex(); f'{h[0:4]}-{h[4:6]}-{h[6:8]}'
C:      sprintf(out, "%02X%02X-%02X-%02X", raw[0],raw[1],raw[2],raw[3]);
Go:     hex.EncodeToString(raw) // then slice
```

12.3 Packed BCD Postcode (3 bytes → 5 digits)

```
Raw: 12 34 50 → hex "123450" → take first 5 chars → "12345"
```

12.4 Multi-Line Name (stringM)

GMPCName (80 bytes) = 3 lines of 30+30+20 bytes, each space-padded. Decode each line, strip, join with space.

12.5 JPEG Extraction

```
1. Read 4000 bytes from File 2
2. Find last occurrence of FF D9 (JPEG EOI marker)
3. Slice at position + 2
4. Save as .jpg
```

12.6 Gender Byte

Offset 0x11B in File 1: ASCII 'L' = Lelaki (Male), 'P' = Perempuan (Female).

13. Complete Extractable Data Reference

Key	Source	Type	Sample
name	File 1	string	ALI BIN ARUMUGAM
gmpc_name	File 1	string	ALI ARUMUGAM
kpt_name	File 1	string	(alias)
ic	File 1	string	010517100416
gender	File 1	string	Male / Female
old_ic	File 1	string	S1234567
dob	File 1	string	2001-05-17
age	Computed	int	23
birth_place	File 1	string	PULAU PINANG
date_issued	File 1	string	2020-02-11
nationality	File 1	string	WARGANEGARA
race	File 1	string	Indian
religion	File 1	string	Hindu
address1	File 4	string	89-7-3 BATU KUNING FLATS
address2	File 4	string	JALAN TAN SRI TEH EWE LIM
address3	File 4	string	(empty if unused)
postcode	File 4	string	11600
city	File 4	string	GEORGETOWN
state	File 4	string	PULAU PINANG
socso	File 5	string	123456789
locality	File 6	string	BANDAR GEORGE TOWN
photo_bytes	File 2	binary	JPEG — save as .jpg
thumb1	File 3	binary	Right thumb (598B, proprietary)
thumb2	File 3	binary	Left thumb (598B, proprietary)

14. Implementation: Python (Full CLI Tool)

14.1 Dependencies

```
pip install pyscard pillow
# Linux: sudo apt install pcscd libpcsclite-dev
```

14.2 Complete Reader Class

The full Python MyKadReader class (from mykad.py) handles: auto-detection of all connected readers, JPN application selection, chunked file reading with 240-byte hardware-safe limit, packed BCD date and postcode decoding, JPEG extraction, age computation, and JSON output.

```
class MyKadReader:
    JPN_AID = "00 A4 04 00 0A A0 00 00 00 74 4A 50 4E 00 10"
    GET_RESPONSE = "00 C0 00 00 05"

    def __init__(self):
        r = readers()
        # Loop all readers to find one with a card inserted
        for reader in r:
            try:
                conn = reader.createConnection()
                conn.connect()
                self.conn = conn; break
            except: continue

    def send(self, apdu_str):
        apdu = toBytes(apdu_str)
        data, sw1, sw2 = self.conn.transmit(apdu)
        return data, sw1, sw2

    def read_file(self, file_no, offset, length):
        result = []; remaining = length; cur = offset
        while remaining > 0:
            chunk = min(remaining, 0xF0)  # 240 byte max
            ll, lh = chunk & 0xFF, (chunk >> 8) & 0xFF
            ol, oh = cur & 0xFF, (cur >> 8) & 0xFF
            self.send(f"C8 32 00 00 05 08 00 00 {ll:02X} {lh:02X}")
            self.send(f"CC 00 00 00 08 {file_no:02X} 00 01 00 {ol:02X} {oh:02X} {ll:02X}
{lh:02X}")
            data, sw1, sw2 = self.send(f"CC 06 00 00 {chunk:02X}")
            if len(data) == 0: raise Exception(f"Read failed SW={sw1:02X}{sw2:02X}")
            result.extend(data)
            remaining -= len(data); cur += len(data)
        return bytes(result)
```

14.3 Reading Personal Data

```
def read_all(self):
    self.select_jpn()
    f1 = self.read_file(0x01, 0x0003, 0x01A5)
    data = {
        "name":    f1[0x00:0x96].decode('ascii','ignore').strip(),
        "ic":      f1[0x10E:0x11B].decode('ascii','ignore').strip(),
        "gender":  "Male" if f1[0x11B:0x11C] == b'L' else "Female",
        "dob":     self.decode_bcd_date(f1[0x124:0x128]),
        # ... (all other fields at documented offsets)
    }
    # Photo: read File 2, find FF D9, slice
    photo = self.read_file(0x02, 0x0003, 0x0FA0)
    data["photo"] = photo[:photo.rfind(b'\xFF\xD9')+2]
    return data
```

14.4 Build to Executable

```
pip install pyinstaller
pyinstaller --onefile mykad_reader.py
# Output: dist/mykad_reader.exe (no Python needed on target)
```

14.5 Performance Optimization (Skip the Photo)

Smart card communication runs over a serial interface (typically 9600–115200 baud). Reading the full JPN application including the 4000-byte JPEG photo takes approximately 3–5 seconds of real physical I/O time. This is not a software bottleneck — it is a hardware limitation of the serial protocol between the reader and the chip.

Read time breakdown:

Operation	Data Size	Approx. Time
Select JPN application	~20 bytes	< 0.1s
Read File 1 (Personal identity)	421 bytes	~0.2s
Read File 4 (Address)	148 bytes	~0.1s
Read File 5 + 6 (SOCSO + Locality)	19 bytes	< 0.1s
Read File 2 (JPEG Photo)	4000 bytes	~2–4s
Read File 3 (Fingerprints)	~1216 bytes	~1–2s
TOTAL (with photo)	~5804 bytes	~3–5s
TOTAL (without photo)	~608 bytes	~0.3–0.5s

Standard UI/UX: Always display a “Reading card — please do not remove…” loading indicator in your frontend. Removing the card mid-read will corrupt the data and may require re-insertion.

Optimization: If your application only needs identity data (e.g. auto-filling a registration form, verifying IC number, or checking age) and does not need the face photo, skip reading File 0x02 entirely. Reading only File 1 + File 4 reduces total read time from ~4 seconds to under 0.5 seconds — an 8× speedup.

Progressive loading (recommended for kiosks): The MyKad Reader Pro server implements this pattern: read personal data + address first (fast), return it to the UI immediately, then read the photo in a second pass. This gives the user instant feedback while the photo loads in the background. See Section 19 (/api/read_card then /api/read_card/photo).

15. Implementation: C / WinSCard (Low-Level)

For embedded systems, kiosks, and performance-critical applications. Uses the native Windows Smart Card API (winscard.h) or PCSC-Lite on Linux.

15.1 Windows (winscard.h)

```
#include <windows.h>
#include <winscard.h>
#include <stdio.h>
#pragma comment(lib, "winscard.lib")

BYTE SELECT_JPN[] = {0x00,0xA4,0x04,0x00,0x0A,
    0xA0,0x00,0x00,0x00,0x74,0x4A,0x50,0x4E,0x00,0x10};
BYTE GET_RESP[] = {0x00,0xC0,0x00,0x00,0x05};

int send_apdu(SCARDHANDLE hCard, BYTE *apdu, DWORD apduLen,
              BYTE *resp, DWORD *respLen) {
    DWORD proto = SCARD_PROTOCOL_T0 | SCARD_PROTOCOL_T1;
    LONG rv = SCardTransmit(hCard, SCARD_PCI_T1,
        apdu, apduLen, NULL, resp, respLen);
    return (rv == SCARD_S_SUCCESS) ? 0 : -1;
}

int read_mykad_file(SCARDHANDLE hCard, BYTE fileNo,
                    WORD offset, WORD length, BYTE *out) {
    BYTE resp[256]; DWORD respLen;
    WORD remaining = length, cur = offset;
    int pos = 0;
    while (remaining > 0) {
        WORD chunk = (remaining > 0xF0) ? 0xF0 : remaining;
        BYTE setLen[] = {0xC8,0x32,0x00,0x00,0x05,0x08,0x00,0x00,
            (BYTE)(chunk&0xFF), (BYTE)(chunk>>8)};
        respLen=sizeof(resp); send_apdu(hCard,setLen,10,resp,&respLen);
        BYTE selInf[] = {0xCC,0x00,0x00,0x00,0x08, fileNo,
            0x00,0x01,0x00, (BYTE)(cur&0xFF),(BYTE)(cur>>8),
            (BYTE)(chunk&0xFF),(BYTE)(chunk>>8)};
        respLen=sizeof(resp); send_apdu(hCard,selInf,13,resp,&respLen);
        BYTE readCmd[] = {0xCC,0x06,0x00,0x00,(BYTE)chunk};
        respLen=sizeof(resp); send_apdu(hCard,readCmd,5,resp,&respLen);
        DWORD dataLen = respLen - 2; /* strip SW1 SW2 */
        memcpy(out + pos, resp, dataLen);
        pos += dataLen; remaining -= dataLen; cur += dataLen;
    }
    return pos;
}
```

15.2 Linux (PCSC-Lite)

Nearly identical to Windows. Replace #include <winscard.h> with <winscard.h> from pcsc-lite. Compile with: gcc mykad.c -o mykad -lpcsclite

16. Implementation: Go (Struct-Based)

The Go implementation uses a struct-based approach for hotel PMS and enterprise integrations. The uploaded Scan_Image_Base.go and Scan_Image_Service.go files show the AI/OCR approach using OpenAI's vision API to extract ID card data from images.

16.1 CardDetails Model

```
type CardDetails struct {
    CardNumber      string `json:"cardNumber"`
    FullName        string `json:"fullName"`
    DateOfBirth     string `json:"dateOfBirth"`
    Gender          string `json:"gender"`
    Nationality     string `json:"nationality"`
    NationalityCode string `json:"nationalityCode"`
    Address         string `json:"address"`
    Country         string `json:"country"`
    CountryCode     string `json:"countryCode"`
    ExpireDate      string `json:"expireDate"`
    CardTypeName    string `json:"cardTypeName"`
    CardType        int    `json:"cardType"`
    ErrorCode       int    `json:"errorCode"`
    FieldError      string `json:"fieldError"`
}
```

16.2 Repository Interface

```
type IDCardScannerRepository interface {
    ScanImage(imageUrl string) (CardDetails, error)
}
```

17. Implementation: Node.js (OpenSC)

The Node.js approach shells out to opensc-tool and parses fixed-column stdout output.

17.1 Install OpenSC

Download from github.com/OpenSC/OpenSC/releases. Place opensc-tool.exe in your project's ./opensc/ directory.

17.2 APDU Sequence

```
const { exec } = require('child_process');
exec('opensc-tool.exe -v ' +
  '-s "00:A4:04:00:0A:A0:00:00:00:74:4A:50:4E:00:10" ' +
  '-s "C8:32:00:00:05:08:00:00:F0:00" ' +
  '-s "CC:00:00:00:08:01:00:01:00:03:00:F0:00" ' +
  '-s "CC:06:00:00:F0" ' +
  // ... additional chunks for File 1 remainder + File 4
  , { cwd: openscPath }, (error, stdout) => {
    const parsed = convertMyKadData(stdout);
  });
```

18. Cross-Language Checksum Validators

Drop-in ISO 7064 Mod 11,2 checksum functions for any backend. Each takes the first 11 digits and returns the expected check digit (or ‘X’ for impossible sequences).

18.1 JavaScript

```
function checksumMod112(first11) {
  const arr = first11.split('').reverse().map(Number);
  const W = i => Math.pow(2, i-1) % 11;
  let sum = 0;
  for (let j = 0; j < 11; j++) sum += arr[j] * W(j+2);
  const cd = (12 - (sum % 11)) % 11;
  return cd === 10 ? 'X' : String(cd);
}
```

18.2 Python

```
def checksum_mod112(first11: str) -> str:
    arr = [int(d) for d in reversed(first11)]
    W = lambda i: pow(2, i-1, 11)
    s = sum(arr[j] * W(j+2) for j in range(11))
    cd = (12 - (s % 11)) % 11
    return 'X' if cd == 10 else str(cd)
```

18.3 C# (.NET)

```
public static string ChecksumMod112(string first11) {
    var arr = first11.Reverse().Select(c => c - '0').ToArray();
    int W(int i) => (int)(Math.Pow(2, i-1) % 11);
    int sum = 0;
    for (int j = 0; j < 11; j++) sum += arr[j] * W(j+2);
    int cd = (12 - (sum % 11)) % 11;
    return cd == 10 ? "X" : cd.ToString();
}
```

18.4 PHP

```
function checksumMod112(string $first11): string {
    $arr = array_reverse(str_split($first11));
    $W = fn($i) => (2 ** ($i - 1)) % 11;
    $sum = 0;
    for ($j = 0; $j < 11; $j++) $sum += (int)$arr[$j] * $W($j + 2);
    $cd = (12 - ($sum % 11)) % 11;
    return $cd === 10 ? 'X' : (string)$cd;
}
```

18.5 Go

```go
func checksumMod112(first11 string) string {
    digits := []int{}
    for i := len(first11)-1; i >= 0; i-- {
        digits = append(digits, int(first11[i]-'0'))
    }
    W := func(i int) int {
        v := 1
        for k := 0; k < i-1; k++ { v = (v * 2) % 11 }
        return v
    }
    sum := 0
    for j := 0; j < 11; j++ { sum += digits[j] * W(j+2) }
    cd := (12 - (sum % 11)) % 11
    if cd == 10 { return "X" }
    return fmt.Sprintf("%d", cd)
}
```

18.6 Java

```java
public static String checksumMod112(String first11) {
    char[] chars = new StringBuilder(first11).reverse().toString().toCharArray();
    int sum = 0;
    for (int j = 0; j < 11; j++) {
        int w = (int)(Math.pow(2, j+1) % 11);
        sum += (chars[j] - '0') * w;
    }
    int cd = (12 - (sum % 11)) % 11;
    return cd == 10 ? "X" : String.valueOf(cd);
}
```

18.7 C (Standard C89/C99 — MSVC and GCC compatible)

```c
char checksum_mod112(const char *first11) {
    int arr[11], i, k, sum = 0;
    for (i = 0; i < 11; i++) arr[i] = first11[10 - i] - '0';

    for (i = 0; i < 11; i++) {
        int w = 1;
        for (k = 0; k < i + 1; k++) w = (w * 2) % 11;
        sum += arr[i] * w;
    }

    int cd = (12 - (sum % 11)) % 11;
    return cd == 10 ? 'X' : (char)('0' + cd);
}
```

18.8 Full Validation Logic (Any Language)

A complete IC number validator checks: (1) exactly 12 digits, (2) valid date (YYMMDD), (3) valid PB code, (4) checksum match, (5) gender parity.

```
function validateIC(ic12) {
  if (ic12.length !== 12 || /\D/.test(ic12)) return {valid:false,reason:'Not 12 digits'};
  const dob = parseDOB(ic12);        // extract and validate date
  const pob = POB_CODES[ic12.slice(6,8)]; // lookup place code
  const expected = checksumMod112(ic12.slice(0,11));
  const actual = ic12[11];
  if (expected === 'X') return {valid:false,reason:'Impossible sequence'};
  if (expected !== actual) return {valid:false,reason:'Checksum mismatch'};
  const genderDigit = parseInt(actual, 10);
  return {valid:true, dob, pob, gender: genderDigit % 2 !== 0 ? 'Male' : 'Female'};
}
```

19. Server API — Flask REST + WebSocket Architecture

The MyKad Reader Pro server (server.py v5.0) provides a full REST API with token-based authentication, auto-scan with card insertion events, progressive reading (personal data first, then photo), WebSocket agent support for multi-station deployments, and SSE real-time card events.

19.1 Architecture

Mode	Description
Standalone	Server reads local USB reader directly. GET /api/read_card triggers read.
Server Mode	Multiple agents connect via WebSocket. POST /api/stations/<n>/scan triggers remote read.

19.2 Authentication

Login with POST /api/auth/login to get a session token. Include X-Token header or ?token= query param on all authenticated endpoints. A permanent API token can be generated for PMS integration.

19.3 Key Endpoints

Method	Path	Auth	Description
POST	/api/auth/login	No	Login, returns session token
GET	/api/status	No	Reader state: offline/idle/card_detected/busy/ready
GET	/api/readers	No	List connected smart card readers
GET	/api/read_card	Yes	Trigger card read (standalone mode)
GET	/api/read_card/photo	Yes	Fetch photo after read_card
GET	/api/records	Yes	List records with filtering
POST	/api/records	Yes	Create or update record
GET	/api/guest/<ic>	Yes	PMS lookup by IC number
GET	/api/export/pdf/<id>	Yes	Export as PDF
GET	/api/export/json	Yes	Export all as JSON
GET	/api/export/excel	Yes	Export all as Excel
GET	/api/card_events	No	SSE stream: real-time card events
POST	/api/stations/<n>/scan	Yes	Trigger remote agent scan (server mode)

19.4 Database Schema (SQLite)

```
CREATE TABLE IF NOT EXISTS records (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    ic_number TEXT, name TEXT, gmpc_name TEXT, kpt_name TEXT,
```

```
    gender TEXT, old_ic TEXT, dob TEXT, age INTEGER,
    birth_place TEXT, date_issued TEXT, nationality TEXT,
    race TEXT, religion TEXT,
    address1 TEXT, address2 TEXT, address3 TEXT,
    postcode TEXT, city TEXT, state TEXT,
    socso TEXT, locality TEXT, photo_b64 TEXT,
    contact_1 TEXT, contact_2 TEXT, contact_3 TEXT,
    email TEXT, custom_json TEXT, notes TEXT,
    raw_dump TEXT, station_name TEXT,
    created_at TEXT, updated_at TEXT
);
```

20. AI/OCR ID Card Scanning Integration

For systems that photograph the physical MyKad instead of reading the chip (e.g. hotel check-in kiosks, mobile apps), AI-powered OCR can extract the printed data.

20.1 Approach (from Scan_Image_Service.go)

The Go implementation sends the card image to OpenAI's GPT-4.1-nano vision model with a structured prompt requesting JSON output. The prompt specifies exact field names, date formats, card type codes, and error handling for blurry or non-ID images.

20.2 Prompt Design

The base prompt instructs the AI to extract: cardNumber, fullName, dateOfBirth (yyyy-mm-dd), gender, nationality, nationalityCode (ISO2), address, country, countryCode (ISO2), expireDate, cardTypeName, cardType (numeric), with error codes for non-ID images (errorCode 1) and blurry images (errorCode 2).

20.3 Card Type Codes

Code	Type
1	ID Card (default, includes MyKad)
2	Driving Licence
3	Passport
11	Residence Permit Card
12	Visa

20.4 Post-Processing

After extraction, the system validates nationality and country codes against a database of ISO2 country codes to ensure standardised values. The Go implementation queries a countries table and maps the AI-returned codes to canonical ISO2 values.

21. Database Schema (Full SQL)

Generic SQL schema compatible with SQLite, PostgreSQL, and MySQL:

```
CREATE TABLE mykad_data (
    record_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    ic_number     VARCHAR(12)   NOT NULL,
    name_1        VARCHAR(150),    -- JPN_OrgName
    name_2        VARCHAR(80),     -- JPN_GMPCName
    name_3        VARCHAR(40),     -- JPN_KPTName
    gender        VARCHAR(10),
    race          VARCHAR(25),
    religion      VARCHAR(11),
    state         VARCHAR(30),
    city          VARCHAR(25),
    address1      VARCHAR(30),
    address2      VARCHAR(30),
    address3      VARCHAR(30),
    postcode      VARCHAR(5),
    contact_no_1  VARCHAR(20),
    dob           DATE,
    nationality   VARCHAR(18),
    old_ic        VARCHAR(8),
    socso         VARCHAR(9),
    locality      VARCHAR(10),
    birth_place   VARCHAR(25),
    date_issued   DATE,
    photo_path    VARCHAR(255),    -- or photo_b64 TEXT for inline
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

22. Security, PDPA Compliance & Best Practices

22.1 Reading: No Auth Required

The contact chip requires NO PIN, NO key. Any PC/SC reader can extract all data. This is by design for identity verification.

22.2 Writing: Heavily Protected

Writing is **BLOCKED** without JPN Perso Card + Triple-DES key. **3 failed attempts = permanently blocked.**

22.3 Data Masking

Always mask IC numbers on frontend displays:

```
Display: 980317-14-***8   (last 4 digits hidden)
Or:      ****17-14-5138   (first 4 digits hidden)
// JavaScript:
const mask = ic => ic.slice(0,8) + '***' + ic.slice(11);
```

22.4 Encryption at Rest

Biometric data (photos, fingerprints) should be encrypted at rest using AES-256. For the database, use SQLCipher (SQLite with AES-256-CBC encryption) or equivalent.

22.5 Memory Management (C/C++)

For low-level implementations, wipe APDU response buffers after processing:

```
// Windows:
SecureZeroMemory(buffer, sizeof(buffer));
// Linux/cross-platform:
explicit_bzero(buffer, sizeof(buffer));
```

22.6 PDPA Compliance (Malaysia)

- Always obtain informed consent before scanning a card.
- Do not store biometric data without explicit consent.
- Implement data retention policies — delete records after business purpose is fulfilled.
- Provide data access and deletion rights to cardholders.
- Log all data access for audit trails.

22.7 TnG RFID Security

The TnG Mifare Classic 1K chip has 16 sectors with separate A/B keys. In 2016, keys were brute-forced. Keys have since been updated. Do NOT interface with TnG without authorisation from Touch ‘n Go Sdn Bhd.

22.8 Checksum Disclaimer

The checksum algorithm is community-researched and NOT officially confirmed by JPN. Do not rely on it as sole validation. A passing checksum does not mean an IC number is assigned to a real person.

23. Reference Resources

23.1 Repositories

Repository	Language	Description
deadboy18/mykad-reader	Mixed	Main project — SDK, reader, server, MDB
beeell1/malaysian-ic-number-validator	HTML/JS	Bryan Lam's IC checksum validator
HakamRaza/Malaysia-MyKad-Reader-Webserver	Node.js	opensc-tool + stdout parsing
amree/mykad-java	Java	javax.smartcardio implementation
yapyeeqiang/mykad-reader	Various	Scan and extract Malaysian IC
firdausramlan/MyKAD-Reader	HTML/JS	Chrome plugin approach
desmondchoon/HTTP-SmartCard-Reader	Various	HTTP-based reader on port 8080

23.2 Standards & References

Resource	URL
JPN MyKad Command Set	http://www.jpn.gov.my/en/informasimykad/mykad-command-set/
ISO 7064 (Checksum)	https://en.wikipedia.org/wiki/ISO/IEC_7064
ISO 7816 (Smart Cards)	https://en.wikipedia.org/wiki/ISO/IEC_7816
OpenSC Project	https://github.com/OpenSC/OpenSC
PC/SC Workgroup	https://pcscworkgroup.com/
pyscard (Python)	https://pyscard.sourceforge.io/
Lowyat APDU Deep Dive	https://forum.lowyat.net/index.php?showtopic=355950
amree's blog	https://amree.dev/2011/12/05/how-to-read-mykad/

END OF DOCUMENT

MyKad Complete Technical Reference v3.0 — April 2026
Author: (deadboy18) | https://github.com/deadboy18/mykad-reader